

# BeMaMaJa

Group Project Report – INFO4271 Modern Search Engines

Matthias Vetter, Marcel John, Janis Weller, Bernhard Fraidel

<https://github.com/BeMaMaJa-Search-Engine/BeMaMaJa-Search-Engine.git>

## Abstract

This project presents a search engine built from scratch for English-language web pages related to Tübingen. It starts with a web crawler that uses a given seed set to retrieve web pages. To avoid crawling undesirable websites, a blacklist is used, and the crawler respects the robots.txt protocol. To improve performance, the crawler uses multiple worker threads. For indexing, the retrieved pages are tokenized, stopwords are removed, and the remaining words are stemmed before constructing the inverted index. The retrieval stage is based on the BM25 algorithm with an integrated spelling correction mechanism. For re-ranking, we combine the BM25 score with field boosts, link counts, PRF, location-aware ranking signals, and soft domain diversification. The frontend provides a modern interface with controllable AI-generated page summaries, explainable ranking signals and direct batch-file upload.

## 1 Data Collection and Crawling

### 1.1 Corpus scope and seeds

Our corpus contains English-language websites related to Tübingen. For the initial seed set, we selected around 50 keywords covering attractions, the university, student life, hospitals, culture, and other city topics. For each keyword, we searched Google for “Tübingen + keyword” and manually recorded the first ten results. Removing duplicate URLs left 449 unique seeds. We then added <https://scells.me/> manually, bringing the final seed set to 450 URLs. The broad range of keywords was intended to produce a diverse corpus rather than focusing only on the most common tourist pages.

### 1.2 Crawler design

We implemented four crawler versions and evaluated each for about 15 hours under the same conditions before selecting the best-performing version for the final crawl. Version 1 followed the architecture from the lecture. It used a prioritized frontier, crawl delays, a custom user agent, and multiple worker threads. It also respected blocked paths and crawl delays from `robots.txt`, while caching each file to avoid repeated downloads. We used `langdetect` instead of website-provided language metadata. Duplicate content was kept during crawling and removed during preprocessing. This increased the size of `raw_pages.json` but avoided slowing down the crawler.

Version 2 retained this design but expanded the regular expression for Tübingen-related pages to cover spelling variations and OCR-like errors such as “Tueeebingen” and “t123bingen”. This allowed relevant pages to be detected even when the city name was misspelled or written in an unusual style.

Version 3 addressed the increasing cost of sorting a large global frontier, which often blocked the worker threads. It split links into a high-priority frontier for links from Tübingen-related pages and a low-priority frontier for links from unrelated pages. Instead of one large URL data structure, each frontier stored domain objects with a queue of URLs and the last access time. The crawler randomly selected a ready domain and processed its next URL in discovery order. Ready domains had an equal chance of being selected regardless of queue size, so a large website with thousands of links could not dominate a smaller domain. Selection depended on the number of domains rather than the total number of URLs, which kept frontier operations efficient even when millions of links were stored. This also allowed the main thread to spend less time managing the frontier and more time supplying work to the workers. The trade-off was losing fine-grained prioritization within a domain: a URL containing “Tübingen” could be processed after less relevant URLs from the same website. However, Version 3 still crawled roughly four to five times as many pages as Versions 1 and 2.

Version 4 added two worker types. Horses selected a random ready domain as before, while Explorers preferred ready domains with only a few URLs left. Since Horses would eventually reach these domains as well, this was expected to have little effect on overall throughput. Its purpose was to discover new domains slightly earlier and improve exploration. We also maintained a domain blacklist containing <https://archive.org>.

### 1.3 Results

We evaluated the crawlers with `scripts/crawler_analysis.py`. After about ten hours, Versions 1 and 2 still found the most unique domains, whereas the later versions achieved almost four times the page throughput. Comparing

Versions 2 and 4, Version 4 collected about 3.57 times as many pages while finding about 4% fewer domains. We accepted this trade-off, especially because the globally sorted frontier of Versions 1 and 2 became slower as it grew.

| Crawler                         | Runtime (min) | Domains | Raw pages | Pages/min |
|---------------------------------|---------------|---------|-----------|-----------|
| Base (V1, blue)                 | 632           | 471     | 5187      | 8.21      |
| Regex (V2, orange)              | 583           | 484     | 5201      | 8.92      |
| Harsh Frontier (V3, red)        | 635           | 416     | 20001     | 31.50     |
| Horses & Explorers (V4, purple) | 628           | 452     | 20000     | 31.85     |

Table 1: Crawler version comparison after about ten hours.

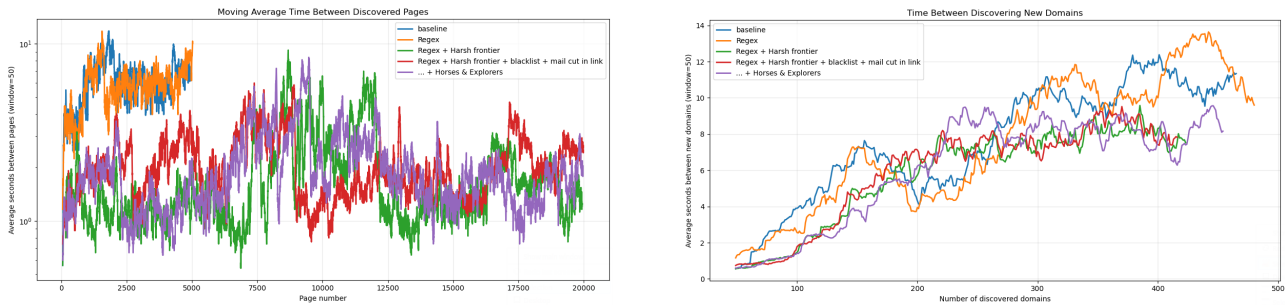


Figure 1: Crawler analysis using a sliding window of 50: average time between finding new pages (left), and time between discovering new domains with the total number of domains found (right). Lower is better.

## 2 Preprocessing and Index Construction

After crawling, exact and near-duplicate pages were removed, reducing the dataset by approximately 30%. Clearly non-English pages were filtered using language detection and a separate check for pages in which at least 25% of the alphabetic characters were non-Latin. Title, headings, and body were processed separately. The text was lowercased, Unicode accents and different spellings of Tübingen were normalized, and it was then tokenized. Finally, stopwords were removed and Porter stemming was applied. Queries use the same preprocessing pipeline. The processed documents were used to construct the inverted index. We also stored document frequencies, field lengths, and the average document length. An internal link graph was built to support link-based re-ranking in later stages.

## 3 Retrieval and Re-ranking

### 3.1 Manual BM25 first stage

BM25 is used for first-stage ranking because it is a standard algorithm in practice. It computes the Inverse Document Frequency (IDF) to give rare words a higher value. Since the search engine is Tübingen-based, the IDF for Tübingen is boosted by a factor of 2. The Term Frequency (TF) is used to give words that appear more often a higher value. The final BM25 score is obtained by summing the contribution of each query term. The parameter  $b$  controls the strength of the document length normalization. The value  $b = 0.75$  is chosen since it is the default value in the literature. The parameter  $k_1$  controls the rate of term saturation. Here, the value  $k_1 = 1.2$  is chosen since it is the default value in the literature. [1]

### 3.2 Query processing

For BM25 and spelling correction, the existing cache is used to load the JSON file. The file contains the documents and their vocabulary. This vocabulary is separated into buckets for later spelling correction. Using the cache provides a significant speedup because the JSON file is loaded and parsed only once. Once the data is loaded, the query tokens are preprocessed in the same way as the indexed documents. The spelling of each query term is then checked. The algorithm first checks whether the term exists in the vocabulary, which is built from the indexed pages and contains stemmed terms. If it exists, the spelling is considered correct and no correction

is needed. Otherwise, the term is treated as a possible spelling mistake, and the algorithm tries to correct it. To avoid changing a word too much, at most one character can be changed in words shorter than five characters and at most two in longer words. The buckets are sorted by first letter and word length, so only suitable candidates have to be evaluated. For further speedup, the algorithm exits early if it is no longer possible to find a better term. Next, the Levenshtein edit distance [2] is computed to measure how similar two words are. This is used to find the best candidate. If two words have the same edit distance, their frequency is used as a tie-breaker, since the assumption is that a user is more likely to search for a more frequently used term. If the edit distance for the best candidate is above the number of characters that are allowed to be changed, the word will not be changed to avoid changing the query too much. After a term has been corrected, the word is stemmed again to ensure the resulting term is the same as without the spelling mistake, thus the search results are also identical. After this, the BM25 score is computed, and the result is then used by the reranking algorithm.

### 3.3 Second-stage innovation

As second-stage innovations, we use multiple signals to rerank our results. The final score is a weighted sum of all these components. (0.60 BM25, 0.20 Field Boost, 0.10 Link Score, 0.05 PRF, 0.05 Tuebingen Centrality, -0.10 Location Penalty)

**Field Boost** increases the score when query terms appear in the title or the headings. Matches in these fields are generally stronger relevance indicators than occurrences in the body. For the computation, the following formulas are used, where  $Q$  is the query tokens,  $T$  represents the title tokens, and  $H$  represents the heading tokens.

$$\text{TitleCoverage} = \frac{|Q \cap T|}{|Q|}$$

$$\text{HeadingCoverage} = \frac{|Q \cap H|}{|Q|}$$

$$\text{FieldBoost} = 0.7 \cdot \text{TitleCoverage} + 0.3 \cdot \text{HeadingCoverage}$$

**PRF** is performed by taking the first 30 BM25 candidates. Only the strongest from each domain is considered. Up to 5 documents are selected. Terms are collected from the titles and headings of these five documents. Query terms, generic terms, and terms that are too rare or too frequent in the corpus are removed. At most three expansion terms are retained.

**Link Score** represents the authority of a document within the crawled collection. It is defined as the number of indexed documents that link to the candidate.

**Tübingen Centrality** measures how strongly a document is focused on Tübingen. It is based on the occurrence of the normalized "tubingen" term in important document fields.

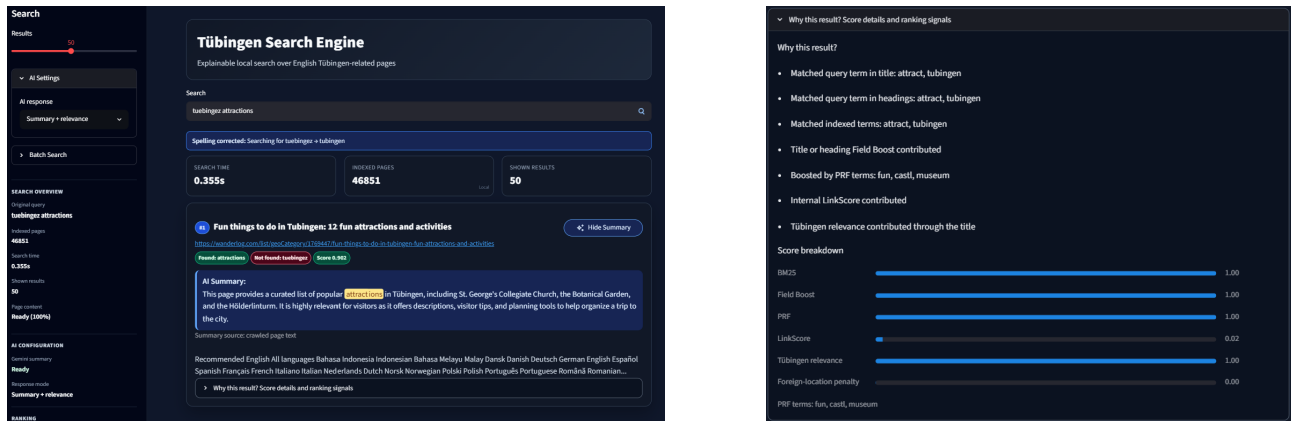
**Foreign Location Penalty** reduces the score of documents that appear to focus on another city, such as Stuttgart, Munich, Heidelberg, or Freiburg. The penalty depends on where the foreign location occurs. Matches in the title receive the strongest penalty, followed by headings and the URL.

### 3.4 Domain diversification

To prevent the result list from being dominated by a single website, we added a soft domain-diversification step after reranking. The first result from each domain keeps its full score. Further results from the same domain are progressively penalized using a factor of 0.93 per repetition. The penalty is limited to a minimum factor of 0.75. We intentionally avoided a strict domain limit because several pages from the same website may still be highly relevant. This approach improves result variety while ensuring that relevant documents remain within the top 100 results used for evaluation.

## 4 Frontend and Hosting

**Frontend innovations.** A good search engine should not only return relevant links, but also help users understand and work with the results. This is why we added several helpful innovations, mainly focused on controllable AI summaries for individual pages and clear insight into why each result appears at its position.



**Figure 1.** Search interface showing spelling correction, Found and Not found query-term badges, and an AI-generated result summary (left), and the expanded ranking explanation with its score breakdown (right).

## 4.1 Interactive interface

**AI summaries.** Each result card has an *AI Summary* button backed by the Gemini API. *Summary + relevance* connects the page summary to the query, *Summary only* gives a neutral overview, and *Custom focus* accepts a short user prompt, for example about opening hours or entrance fees. A custom animation is shown during AI generation. The summaries use cached page content, meaning the crawled body text from the raw-page data; this content is not used for retrieval or ranking. When unavailable, the snippet is used, and if the available text is too short, the page can be fetched live. If Gemini is unavailable or not configured, a local extractive summary is shown instead. Summaries are cached to avoid unnecessary API calls.

**Explainable ranking.** Each result card directly shows its final ranking score. A *Found* badge lists the exact matched query terms, while *Not found* lists the remaining terms. Applied spelling corrections are shown above the results so users know which terms were used for retrieval. For further detail, the expandable *Why this result?* view shows textual match evidence and a normalized breakdown of every active component: BM25, Field Boost, PRF, LinkScore, Tübingen relevance and foreign-location penalty.

**Controls and fallback modes.** Users can select between 5 and 100 displayed results. The sidebar shows search time, index size, active ranking components, data source (Supabase or local), Gemini status (ready or not configured), and page-content loading progress (crawled body text for AI summaries). The interface accepts batch-file uploads and returns the retrieved results as a downloadable `results.tsv`. Runtime data normally comes from a private Supabase bucket. Starting with `--local` disables Supabase, and local index files provide a fallback if Supabase cannot be used. These options keep the service reliable: if one feature or external service fails, the rest of the search continues to work so the user notices as little of the failure as possible.

## 4.2 Hosting and resource management

When a user opens the hosted version, DuckDNS resolves the domain to the public IP of the Oracle VM. Nginx, the reverse proxy, forwards the browser request to Streamlit on the same VM. At startup, Streamlit downloads the compressed index and raw-page shards from Supabase Storage, caches the index in RAM, and loads raw-page content gradually for AI summaries. It reuses the cached index for retrieval and sends summary requests to Gemini. The API key stays on the server; local use requires a separate key.

**Index caching.** Caching the loaded index was our largest runtime improvement: it remains in RAM across Streamlit reruns and is reused for all searches. This reduced search times from about ten seconds to usually below one second.

**Startup memory.** As the data grew, combining downloaded shards in RAM caused a large startup peak. To avoid this, downloaded shards are streamed to a disk cache instead of being combined in RAM. The index is then loaded into RAM and cached, so users can search as soon as it is ready. Page content, which is only needed for AI summaries, is processed gradually in the background. Loading all raw pages therefore takes longer, but this avoids the RAM peak and improves scalability.

## References

- [1] Stephen Robertson and Hugo Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4):333–389, 2009.
- [2] Vladimir I. Levenshtein. *Binary Codes Capable of Correcting Deletions, Insertions and Reversals*. Soviet Physics Doklady, 10(8):707–710, 1966.
- [3] Streamlit Inc. Streamlit Documentation. <https://docs.streamlit.io/>. Accessed: August 5, 2026.
- [4] Oracle Corporation. Oracle Cloud <https://docs.oracle.com/>. Accessed: August 5, 2026.
- [5] Supabase. *Storage*. <https://supabase.com/docs/guides/storage>. Accessed: August 5, 2026.